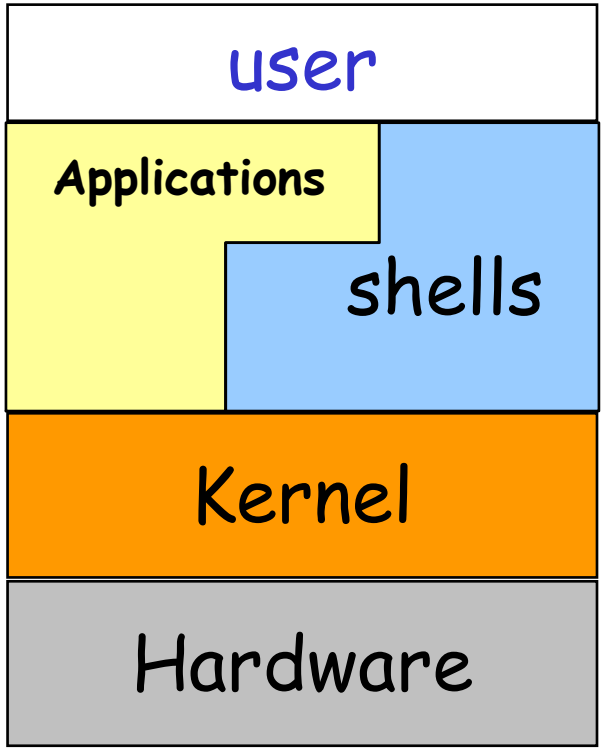
Σενάρια φλοιού
(Shell scripts)

Φλοιός (Shell)

- Είναι το πρόγραμμα που διερμηνεύει (interpreter) τις εντολές που εισάγονται από το πληκτρολόγιο.
- Οι εντολές είναι δυνατόν να τοποθετηθούν σε ένα αρχείο που ονομάζεται **shell script**.
- Κάθε φορά που εισάγεται από το πληκτρολόγιο μια γραμμή το Λ.Σ. χρησιμοποιεί ένα τμήμα του για να αναλύσει το περιεχόμενο της γραμμής, τον **command line processor (CLP)**. Στο UNIX ο CLP είναι εξ ολοκλήρου διαχωρισμένος από το υπόλοιπο Λ.Σ.



- Δηλαδή ο CLP γράφεται ως ξεχωριστό πρόγραμμα και κάθε χρήστης επικοινωνεί με ένα αντίγραφο αυτού του προγράμματος. Αυτό το πρόγραμμα από μόνο του είναι ο φλοιός (SHELL).
- Σε κάθε σύστημα UNIX υπάρχουν τουλάχιστον δύο διαθέσιμοι φλοιοί : **Bourne shell** (**sh**) και **C-shell** (**csh**) (αναπτύχθηκε στα πλαίσια του BSD Unix).
- Άλλοι φλοιοί :
  - **Korn shell (ksh)**
  - **terminal-based C shell (tcsh)**
  - **Bourne again shell (bash) – είναι ο εξ ορισμού φλοιός σε συστήματα linux**
- Ανεξάρτητα από τον φλοιό που χρησιμοποιείται, όλοι έχουν ως στόχο να παρέχουν στο UNIX μια διεπαφή χρήστη (user interface).

Χαρακτηριστικά φλοιών

- Διερμηνεία (interpretation) και επεξεργασία της γραμμής εντολών
- Δεσμευμένες λέξεις
- Μετα-χαρακτήρες φλοιού (wild cards)
- Πρόσβαση και διαχείριση των εντολών των προγραμμάτων φλοιού
- Διαχείριση αρχείων :ανακατεύθυνση εισόδου / εξόδου και διασωληνώσεις
- Υποστήριξη μεταβλητών
- Δημιουργία σεναρίων φλοιού (shell scripts)
- Προγραμματισμός φλοιού (Shell programming)

Αναζήτηση του shell που χρησιμοποιούμε

- **Από το prompt του συστήματος ($, %)**
- **Μέσω της εντολής ps (processes)**

  $ ps –ef : παράγει πληροφορίες για όλες τις διεργασίες (e) σε πλήρη λίστα (f)

- **με χρήση μεταβλητών περιβάλλοντος:**

  echo $SHELL

  /bin/bash

Επιλέγοντας νέο shell

$ ls -lF /bin/*sh* (αναζήτηση των διαθέσιμων shells)

```
-rwxr-xr-x    1 root     root        92444 Φεβ  6  2003 /bin/ash
-rwxr-xr-x    1 root     root       492968 Φεβ  6  2003 /bin/ash.static
-rwxr-xr-x    1 root     root       626028 Απρ  9  2003 /bin/bash
lrwxrwxrwx    1 root     root            4 Ιούλ 7  2004 /bin/bash2 -> bash
lrwxrwxrwx    1 root     root            3 Ιούλ 7  2004 /bin/bsh -> ash
lrwxrwxrwx    1 root     root            4 Ιούλ 7  2004 /bin/csh -> tcsh
lrwxrwxrwx    1 root     root            4 Ιούλ 7  2004 /bin/sh -> bash
-rwxr-xr-x    1 root     root       316276 Ιαν 25  2003 /bin/tcsh
```

$ chsh (αλλαγή φλοιού)

Changing shell for studin
Old shell: /bin/csh
New shell: _

Είσοδος – έξοδος - διασωλήνωση

Είσοδος - έξοδος

- Every program run under Unix is provided with three I/O *streams*
- *stdin*, *stdout* and *stderr*



Είσοδος - έξοδος

- *Σε όλες τις γλώσσες προγραμματισμού υπάρχουν μεταβλητές που αντιστοιχούν σε αυτά τα κανάλια ειδόδου-εξόδου:*
  - *C: stdin, stdout* and *stderr*
  - *C++: cin, cout, cerr*
  - *Perl: STDOUT, STDIN, STDERR*
  - *Java: System.in, System.out, System.err*
  - *……*

Είσοδος - έξοδος

- *Σε κάποιες γλώσσες, όταν παραλείπεται η αναφορά σε αρχείο εννοείται το stdin όταν πρόκειται για ανάγνωση και το stdout όταν πρόκειται για εγγραφή. Πχ:*
  - *C: εγγραφή:*

    fprintf(stdout,"TEST\n");

    printf("TEST\n");

    ανάγνωση:

    fscanf(stdin,"%s",tmp);

    scanf("%s",tmp);

Είσοδος - έξοδος

- *C: άνοιγμα αρχείου:*

```
FILE *f;
f=fopen("/etc/passwd","r");
int fd;
fd = fileno(f);
```

Διαφορετικα:

```
int fd;
fd=open("/etc/passwd",O_RDONLY);
```

(Το fd θα είναι πάντα >=3)

- Μπορούμε να αναφερθούμε σε ένα αρχείο είτε με τον "file pointer" είτε με τον "file descriptor".

Είσοδος - έξοδος

- Ο "file descriptor" είναι ένας integer που μας δίνει την θέση του αρχείου στον πίνακα με τα ανοιχτά αρχεία της διεργασίας.
  - Μια διεργασία έχει 3 ανοιχτά αρχεία τα οποία βρίσκονται στις θέσεις 0,1,2 του πίνακα ανοιχτών αρχείων:

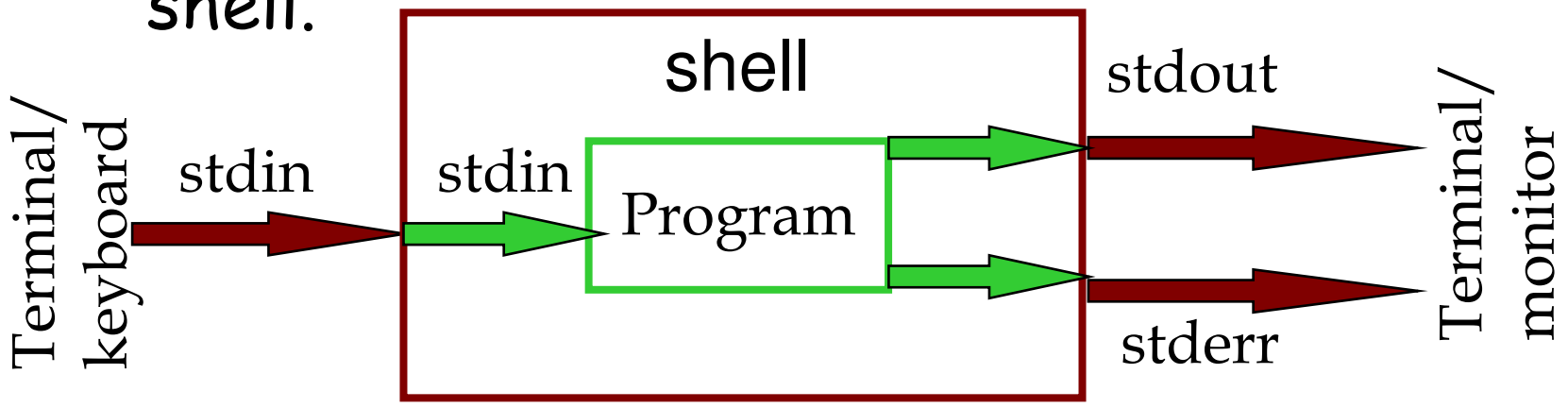


Το λειτουργικό κρατάει διάφορες πληροφορίες για τα ανοιχτά αρχεία όπως:

- Θέση του κέρσορα στο αρχείο
- Mode κατάστασης (read, write, …)
- ….

Είσοδος - έξοδος

- Κάθε διεργασία κληρονομεί από την πατρική της τον πίνακα ανοιχτών αρχείων (τις 3 πρώτες θέσεις με την exec(), όλο τον πίνακα με την fork())
- Ένα πρόγραμμα που εκτελείται από ένα shell, θα κληρονομήσει τα stdin, stdout, stderr του shell.

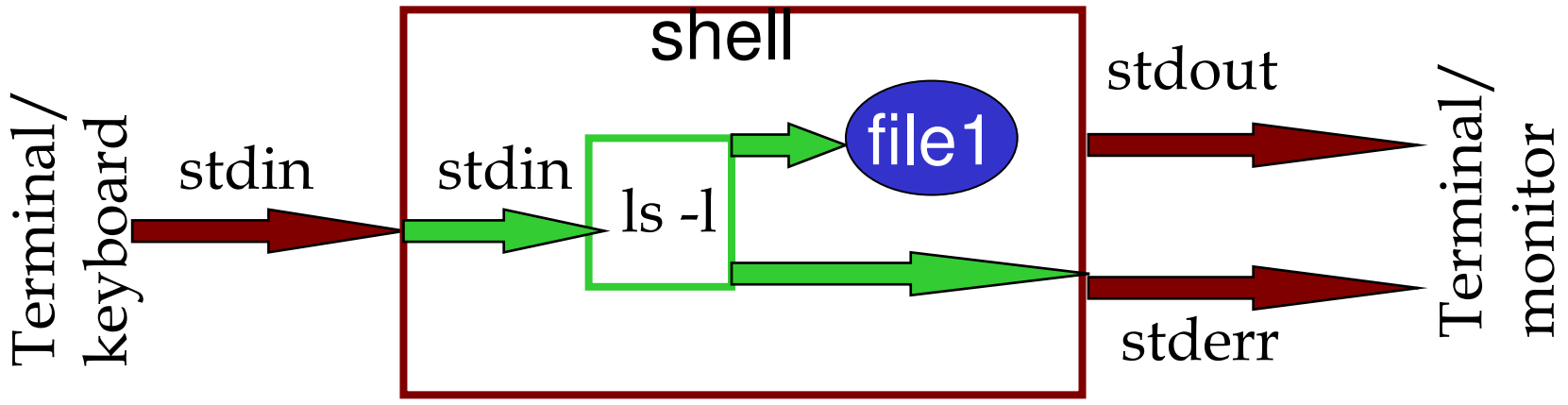


Είσοδος - έξοδος

- Μπορούμε να δώσουμε «οδηγία» στο shell να αλλάξει τα κανάλια IO της διεργασίας που θα δημιουργήσει (του προγράμματος που θα τρέξουμε)

$ ls –l > file1

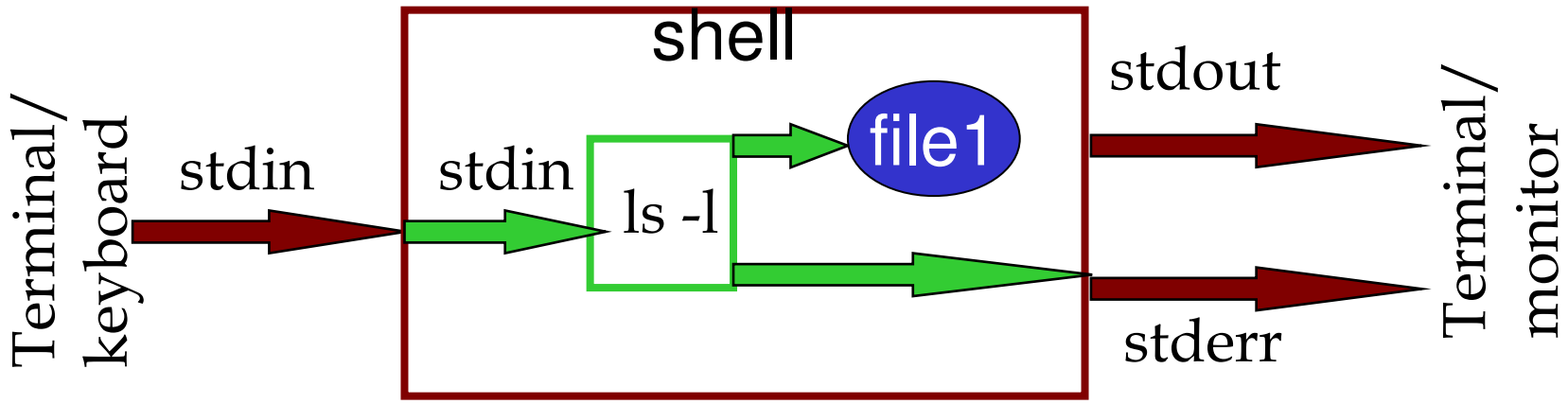
Η ls δεν θα εμφανίσει τίποτα στην οθόνη, αλλά η έξοδός της θα αποθηκευτεί στο αρχείο file1



Είσοδος - έξοδος

$ ls –l > file1

Η ls δεν θα εμφανίσει τίποτα στην οθόνη, αλλά η έξοδός της θα αποθηκευτεί στο αρχείο file1



Αν όμως η ls εμφανίσει μήνυμα λάθους, τότε αυτό θα σταλεί στο "stderr" και άρα στην οθόνη.

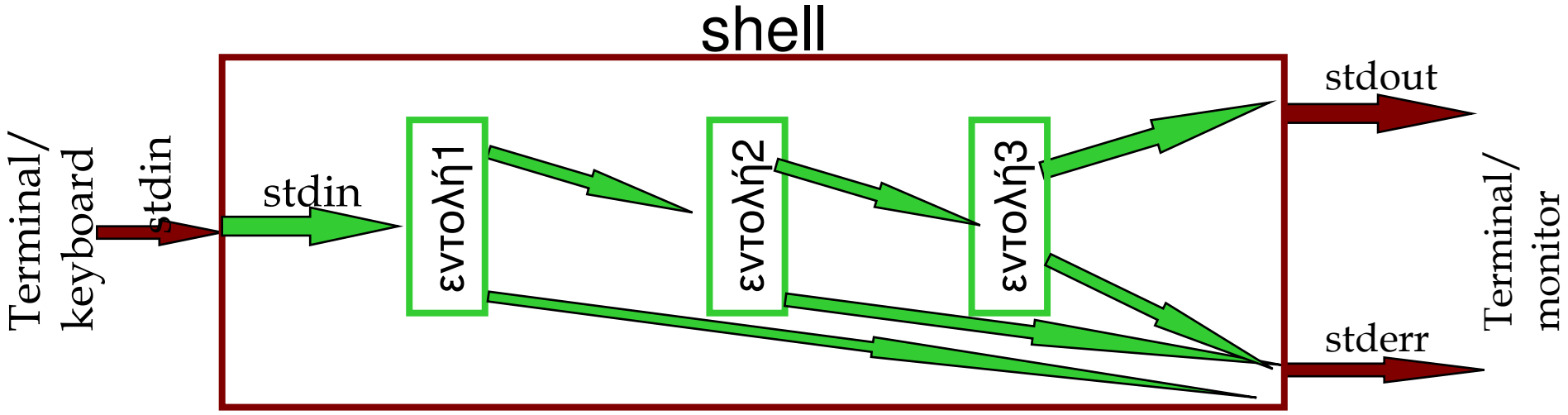
Πχ:

```
ls –l /frag > file1
Cannot access /frag: No such file or directory
```

Είσοδος - έξοδος

- Ειδικοί χαρακτήρες για ορισμό εισόδου/εξόδου από/σε αρχεία:
  - Εντολή > αρχείο (Βάλε stdout στο αρχείο, σβήσε ό,τι περιείχε το αρχείο)
  - Εντολή >> αρχείο (Βάλε stdout στο αρχείο μετά από αυτά που περιείχε το αρχείο (append))
  - Εντολή < αρχείο (Πάρε stdin από το αρχείο)
- Μπορούμε να χρησιμοποιήσουμε συγχρόνως τα stdin και stdout:
  - Εντολή < file_in > file_out

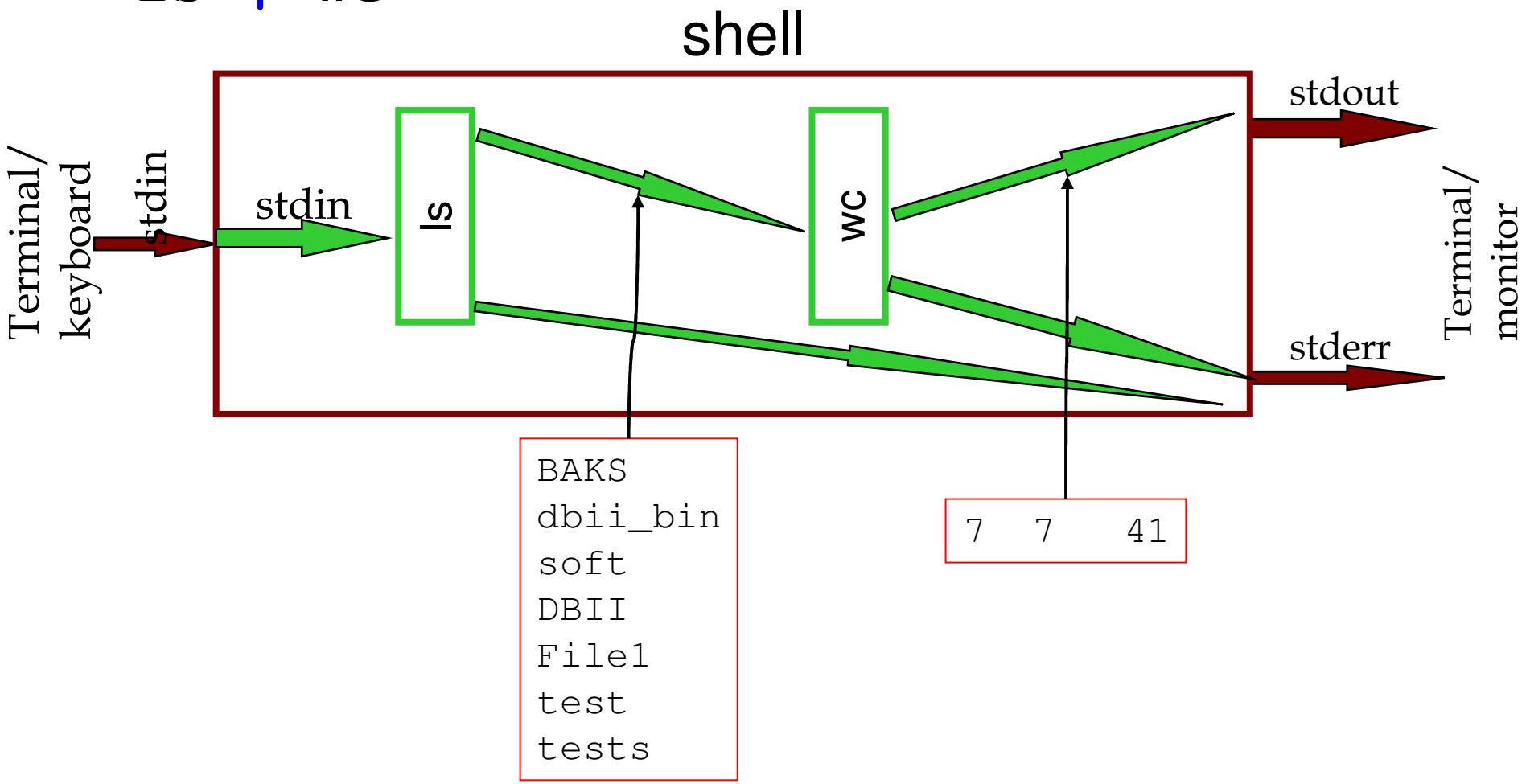
Είσοδος – έξοδος - διασωλήνωση




```
εντολή1 | εντολή2 | εντολή3
```

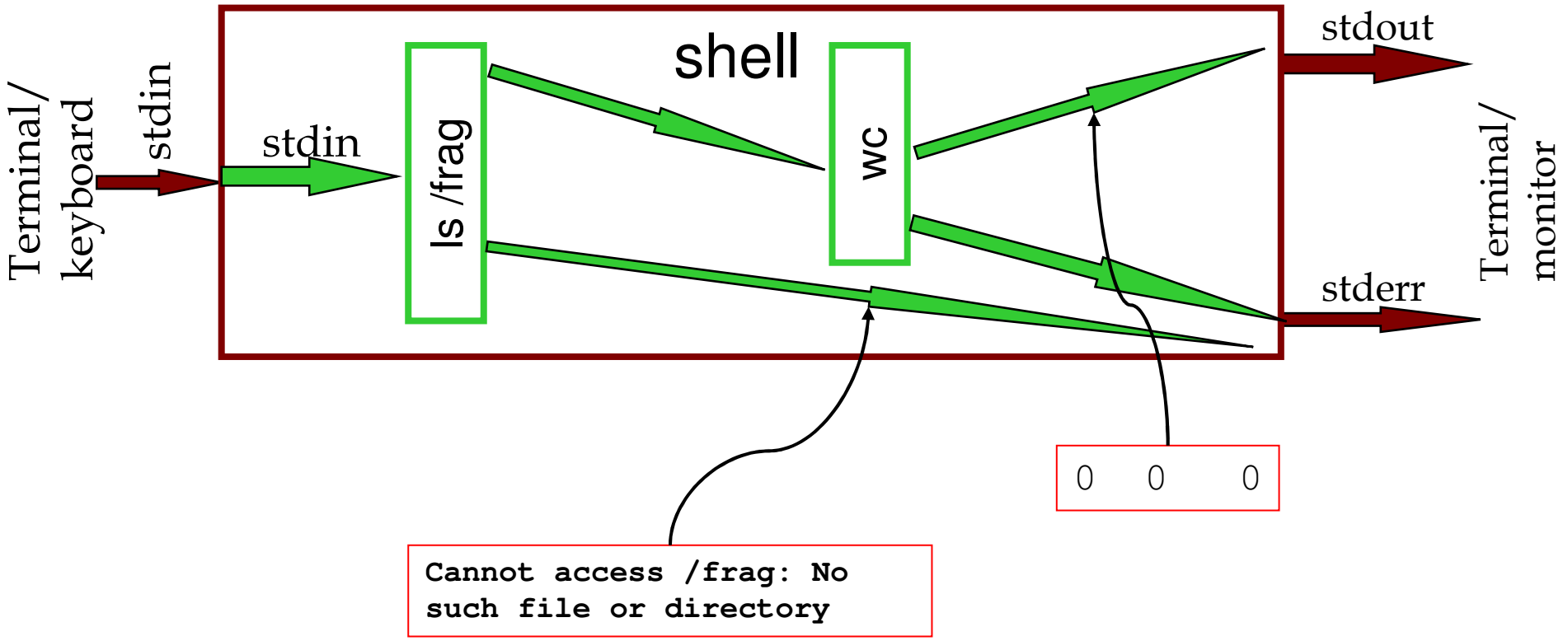
Είσοδος - έξοδος - διασωλήνωση

`ls | wc`



Είσοδος - έξοδος - διασωλήνωση

```
bash-2.05a$ ls /frag | wc
Cannot access /frag: No such file or directory
                0               0               0
bash-2.05a$
```



Είσοδος – έξοδος - διασωλήνωση

```
bash-2.05a$ ls;ls -l
cc
list
total 16
-rw-r--r--    1 frag it           314 Jan 11  2003 cc
-rw-r--r--    1 frag it           183 Jan  8  2003 list
bash-2.05a$
```

```
bash-2.05a$ ls;ls -l > /tmp/test
cc
list
bash-2.05a$ cat /tmp/test
total 16
-rw-r--r--    1 frag it           314 Jan 11  2003 cc
-rw-r--r--    1 frag it           183 Jan  8  2003 list
bash-2.05a$
```

Ο διαχωρισμός εντολών γίνεται με το <enter> ή το ;

Κάθε εντολή είναι ανεξάρτητη από την προηγούμενη

Είσοδος – έξοδος - διασωλήνωση

Αν θέλουμε να ανακατευθύνουμε την έξοδο πολλών εντολών πρέπει να κάνουμε «ομαδοποίηση»

```
bash-2.05a$ (ls;ls -l) > /tmp/test
bash-2.05a$ cat /tmp/test
cc
list
total 16
-rw-r--r--    1 frag it           314 Jan 11  2003 cc
-rw-r--r--    1 frag it           183 Jan  8  2003 list
```

Είσοδος – έξοδος - διασωλήνωση

Αν θέλουμε να ανακατευθύνουμε την έξοδο πολλών εντολών πρέπει να κάνουμε «ομαδοποίηση»

```
bash-2.05a$ ls ; ls -l |wc
cc
list
              3              20             127
bash-2.05a$
```

```
bash-2.05a$ (ls;ls -l ) | wc
              5              22             135
bash-2.05a$
```

Με την ομαδοποίηση τρέχουν οι εντολές της «ομάδας».

Είσοδος – έξοδος - διασωλήνωση

Υπάρχει ένα ειδικό αρχείο στο οποίο μπορούμε να στείλουμε δεδομένα που θέλουμε να «πετάξουμε»

```
bash-2.05a$ ls -l > /dev/null
bash-2.05a$
```

Το αρχείο αυτό «καταπίνει» όλα τα δεδομένα που αποθηκεύονται σε αυτό.

Στο πάνω παράδειγμα δεν θα βρούμε ποτέ/πουθενά την έξοδο της εντολής.

shell

Το shell είναι ένα πρόγραμμα που διαβάζει δεδομένα από το stdin, τα οποία τα θεωρεί εντολές και τις εκτελεί.

Αν δεν ορίσουμε διαφορετικά το stdin ενός shell είναι το τερματικό

Μπορούμε όμως να δώσουμε είσοδο σε ένα shell από αρχείο, όπως σε κάθε άλλο πρόγραμμα.

```
bash-2.05a$ cat file1
ls -l
whoami
date
bash-2.05a$ sh < file1
total 24
-rw-r--r--    1 frag it          314 Jan 11  2003 cc
-rw-r--r--    1 frag it           18 Nov 20 15:22 file1
-rw-r--r--    1 frag it          183 Jan  8  2003 list
frag
Mon Nov 20 15:23:05 EET 2006
bash-2.05a$
```

shell

Η εντολή sh δέχεται ορίσματα. Αν δεχτεί όρισμα όνομα αρχείου, τότε διαβάζει το αρχείο και θεωρεί τα περιεχόμενά του ως εντολές προς εκτέλεση.

shell

- Όταν ένα αρχείο περιέχει text το οποίο είναι εντολές, τότε αυτό το αρχείο ονομάζεται script.
- Όταν το αρχείο περιέχει εντολές για το shell, ονομάζεται shell script
- Στο UNIX τα scripts χρησιμοποιούνται συχνά. Για να μην χρειάζεται κάθε φορά να βρίσκει ο χρήστης για ποιο «πρόγραμμα» (ή shell) είναι οι εντολές που περιέχονται σε ένα script, υπάρχει η σύμβαση στην πρώτη γραμμή του αρχείου να περιέχεται το όνομα της «εντολής» που πρέπει να εκτελέσει το script.
  Η πρώτη γραμμή ενός script πρέπει να είναι:
  #!εντολή
- Για ένα shell script η πρώτη γραμμή πρέπει να είναι:
  #!/bin/sh
- Ο χαρακτήρας # σηματοδοτεί σχόλιο
- Όταν όμως βρίσκεται στην πρώτη γραμμή ενός αρχείου και ακολουθείται από το !, σημαίνει πως αυτό που ακολουθεί είναι η εντολή προς εκτέλεση.

shell

- Έστω ένα αρχείο με όνομα file1 που περιέχει:

#!/path/command
Lala
Foo

- Όταν προσπαθήσουμε να «εκτελέσουμε» το file1, τότε το UNIX θα αναγνωρίσει ότι δεν πρόκειται για binary αρχείο αλλά για text αρχείο.
- Θα διαβάσει την πρώτη γραμμή και θα εκτελέσει την εντολή:

/path/command file1

- Σε ένα shell script (έστω με όνομα αρχείου file1) βάζουμε:

#!/bin/sh
Εντολές

- Όταν προσπαθήσουμε να το εκτελέσουμε το UNIX θα βρεί το #! Και θα εκτελέσει την εντολή:

/bin/sh file1

shell

- Ένα text αρχείο (script) για να μπορέσουμε να το εκτελέσουμε πρέπει να έχουμε άδεια πρόσβασης "execute".
- Συνεπώς όταν δημιουργούμε scripts, πρέπει να τους δώσουμε την άδεια πρόσβασης "execute", η οποία όπως έχουμε δει δεν τίθεται από μόνη της όταν δημιουργούμε αρχεία.
- Αν το αρχείο δεν έχει άδεια "execute" θα πάρουμε το μήνυμα:

```
./file1
./file1: Permission denied.
ls -l
total 24
-rw-r--r--    1 file it          314 Jan 11  2003 cc
-rw-------    1 file it           28 Nov 20 18:40 file1
-rw-r--r--    1 file it          183 Jan  8  2003 list
```

shell

- Εφόσον σε ένα script έχουμε άδεια "execute" όταν το εκτελούμε διαβάζει το λειτουργικό την πρώτη γραμμή κειμένου για να βρει ποιος είναι ο interpreter. Αφού αναγνωρίσει ποιος είναι ο interpreter (πχ /bin/sh) καλεί την εντολή:
  /bin/sh file1
  Ο interpreter αναγνωρίζει ότι δόθηκε όρισμα όνομα αρχείου και προσπαθεί να το ανοίξει για ανάγνωση.
- Αν δεν έχουμε δικαίωμα ανάγνωσης στο αρχείο τότε θα πάρουμε το μήνυμα λάθους:

```
./file1
/bin/sh: ./file1: cannot open: Permission denied
/bin/sh file1
/bin/sh: file1: cannot open: Permission denied
$ls -l
total 24
-rw-r--r--    1 frag it          314 Jan 11  2003 cc
--wx------    1 frag it           28 Nov 20 18:48 file1
-rw-r--r--    1 frag it          183 Jan  8  2003 list
```

- Η πρώτη γραμμή στο script ορίζει το κέλυφος που θα χρησιμοποιηθεί για την εκτέλεση των εντολών που περιέχει το σενάριο (το ! ακολουθεί αμέσως μετά το # χωρίς ενδιάμεσα κενά) :

#!/bin/sh Bourne shell

#!/bin/csh C-shell

#!/bin/tcsh TC-shell

#!/bin/bash BASH shell

#!/usr/bin/perl Perl script

- Όλα τα κελύφη παρέχουν την επιλογή **-v** για να εμφανίζεται στην οθόνη κάθε γραμμή του σεναρίου καθώς αυτό διαβάζεται και την επιλογή **-x** για να εμφανίζονται οι εντολές, καθώς εκτελούνται.
- Οι επιλογές αυτές είναι πολύ χρήσιμες για τη συντακτική ανάλυση των εντολών.

```
bash-2.05a$ cat file1
#!/bin/sh -v
whoami # dixnei poios eimai
#Twra typonoyme tin hmeromhnia
date
bash-2.05a$ ./file1
#!/bin/sh -v
whoami # dixnei poios eimai
frag
#Twra typonoyme tin hmeromhnia
date
Mon Nov 20 19:16:30 EET 2006
```

```
bash-2.05a$ cat file1
#!/bin/sh -x
whoami # dixnei poios eimai
#Twra typonoyme tin hmeromhnia
date
bash-2.05a$ ./file1
+ whoami
frag
+ date
Mon Nov 20 19:17:13 EET 2006
```

shell

- Για να εκτελέσουμε ένα πρόγραμμα ή script πρέπει να αναφερθούμε στο όνομα αρχείου είτε με σχετική είτε με απόλυτη διαδρομή. Πχ:

```
$cat file1
#!/bin/sh
whoami
date
$./file1
frag
Mon Nov 20 18:54:36 EET 2006
$~/test/file1
frag
Mon Nov 20 18:54:42 EET 2006
$pwd
/home/frag/test
```

shell

- Αν δεν δώσουμε διαδρομή για το πρόγραμμα που προσπαθούμε να εκτελέσουμε, το shell δεν ψάχνει την εντολή (αρχείο που θέλουμε να εκτελέσουμε) στον τρέχον κατάλογο.

```
bash-2.05a$ ./file1
frag
Mon Nov 20 18:57:17 EET 2006
bash-2.05a$ file1
bash: file1: command not found
```

shell

- Το shell ψάχνει την εντολή προς εκτέλεση σε ένα σύνολο από καταλόγους που έχουν οριστεί στην μεταβλητή περιβάλλοντος $PATH.

```
bash-2.05a$ file1
bash: file1: command not found
bash-2.05a$ echo $PATH
/usr/freeware/bin:/usr/local/bin:/usr/sbin:/usr/bsd:/sbin:/usr
   /bin:/usr/bin/X11:/usr/freeware/bin:/usr/local/bin:/usr/jav
   a/bin:/usr/freeware/bin:/usr/local/bin:/usr/local/addons/po
   stgre/bin:/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin:/home
   /frag/bin:/usr/sbin:/sbin:/etc:/usr/local/mysql/bin:/usr/et
   c
bash-2.05a$
```

shell

- Αν θέλουμε το shell να ψάχνει την εντολή προς εκτέλεση και στον τρέχον κατάλογο, πρέπει αυτόν να τον προσθέσουμε στο $PATH.

```
PATH=/usr/freeware/bin:/usr/local/bin:/usr/sbin:/usr/bsd:/sb
   in:/usr/bin:/usr/bin/X11:/usr/freeware/bin:/usr/local/bin:/
   usr/java/bin:/usr/freeware/bin:/usr/local/bin:/usr/local/ad
   dons/postgre/bin:/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bi
   n:/home/asidirop/bin:/usr/sbin:/sbin:/etc:/usr/local/mysql/
   bin:/usr/etc:/home/frag/test
export PATH
```

Ή

```
PATH=$PATH:/home/frag/test/
export PATH
```

Το περιβάλλον φλοιού (shell environment)

$ env

```
TERM=xterm
SHELL=/usr/local/bin/bash
SSH_CLIENT=213.7.220.254 50258 22
SSH_TTY=/dev/pts/56
USER=frag
MAIL=/var/mail/frag
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
PWD=/home/frag/PA_PAPER/FIG_2
LANG=en_US.UTF-8
SHLVL=1
HOME=/home/frag
LOGNAME=frag
```